Object-Oriented Programming with C++

Unit 4: Files and Streams — Detailed Practical Code Book

Programme: S.Y. B.Tech. Artificial Intelligence and Data Science

Semester: III

Course: Object-Oriented Programming with C++

Course Code: ADPC303

Unit: 4 — Files and Streams

Language Standard: C++17 or later

Code Book Purpose

This practical code book provides detailed, step-by-step, compilable C++ programs for Unit 4: Files and Streams.

The unit covers:

- Introduction to file handling.
- Types of files.
- Streams and header files.
- File operations.
- Opening files.
- Reading from files.
- Writing to files.
- File pointers and navigation.
- Error handling.

Learning Outcomes

After completing this code book, students should be able to:

1. Create, open, close, read, write, and append text files.
2. Use `ifstream`, `ofstream`, and `fstream` correctly.
3. Check file-opening and file-operation errors.
4. Process files line by line and word by word.
5. Store and retrieve structured records.
6. Use file pointers with `seekg()`, `seekp()`, `tellg()`, and `tellp()`.
7. Work with binary files using `read()` and `write()`.
8. Build basic file-based C++ applications.

How to Use This Code Book

For every Concept:

1. Read the aim.
2. Understand the concept and syntax.
3. Type the program yourself.
4. Compile and run it.
5. Observe the file created in the program folder.
6. Compare your output with expected output.
7. Modify the program as suggested.
8. Prepare viva answers.

Compilation Commands

Linux/macOS

```
g++ -std=c++17 filename.cpp -o program
./program
```

Windows with MinGW

```
g++ -std=c++17 filename.cpp -o program.exe
program.exe
```

Run the executable from a folder where you have permission to create and modify files. The sample files in this book will be created in the current working directory.

Index

Exp. No.	Program title	Main concept
1	Write text to a file	ofstream, open(), close()
2	Read a file line by line	ifstream, getline()
3	Append data to a file	ios::app
4	Copy one file into another	File reading and writing
5	Count lines, words, and characters	File processing
6	Search a word in a file	Text search
7	Store student records in a text file	Structured text records
8	Read and search student records	File parsing
9	Update a record using a temporary file	File update workflow
10	File pointer navigation	seekg(), seekp(), tellg(), tellp()
11	Binary file record writing/reading	write(), read()
12	Random access in a binary file	Record navigation
13	File error handling	fail(), eof(), bad(), good()

14	File statistics mini-project	Text analysis
15	Student record manager mini-project	File-based CRUD operations
16	Library record mini-project	Object-oriented file application

File Streams at a Glance

Stream class	Header	Main use
std::ifstream	<fstream>	Read from a file
std::ofstream	<fstream>	Write to a file
std::fstream	<fstream>	Read and write using the same stream

Common file modes

Mode	Meaning
std::ios::in	Open for reading
std::ios::out	Open for writing
std::ios::app	Append data at the end of the file
std::ios::ate	Open and initially move to the end; later seeking is allowed
std::ios::trunc	Discard existing file content when opening for output
std::ios::binary	Open file in binary mode

Concept 1: Write Text to a File

Aim

To create a text file and write data into it using `std::ofstream`.

Concept

`std::ofstream` creates or opens a file for writing. If the file already exists, opening with the default output mode usually replaces its content.

Program

```
#include <fstream>
#include <iostream>

int main() {
  std::ofstream outputFile("message.txt");

  if (!outputFile) {
    std::cerr << "Error: Could not create message.txt\n";
    return 1;
  }

  outputFile << "Welcome to C++ File Handling\n";
```

```
    outputFile << "This is the first line written to a file.\n";
    outputFile << "Files store data permanently.\n";

    outputFile.close();

    std::cout << "Data written successfully to message.txt\n";
    return 0;
}
```

Expected Console Output

```
Data written successfully to message.txt
```

Expected File Content: message.txt

```
Welcome to C++ File Handling
This is the first line written to a file.
Files store data permanently.
```

Key Learning

- `std::ofstream outputFile("message.txt");` opens a file for writing.
- Always verify whether the file opened successfully.
- `close()` releases the file resource. The destructor also closes the file automatically, but explicit closing is often clear in beginner programs.

Modification Task

Accept three lines from the user and write them to `notes.txt`.

Viva Questions

1. Which header is required for file streams?
2. What is the use of `ofstream`?
3. What happens if an existing file is opened in normal output mode?

Concept 2: Read a File Line by Line

Aim

To read and display the content of a text file using `std::ifstream`.

Program

```
#include <fstream>
#include <iostream>
#include <string>
```

```
int main() {
   std::ifstream inputFile("message.txt");

   if (!inputFile) {
      std::cerr << "Error: Could not open message.txt\n";
      return 1;
   }

   std::string line;

   std::cout << "File Content:\n";
   while (std::getline(inputFile, line)) {
      std::cout << line << '\n';
   }

   inputFile.close();
   return 0;
}
```

Expected Console Output

```
File Content:
Welcome to C++ File Handling
This is the first line written to a file.
Files store data permanently.
```

Key Learning

- `std::getline()` reads a complete line, including spaces.
- The loop continues while line reading succeeds.
- Do not use `while (!inputFile.eof())` for normal line reading; it can cause incorrect final processing.

Modification Task

Display line numbers before every line read from the file.

Concept 3: Append Data to a File

Aim

To add new content at the end of an existing file.

Concept

`std::ios::app` ensures that all new output is placed at the end of the file.

Program

```
#include <fstream>
#include <iostream>

int main() {
  std::ofstream outputFile("message.txt", std::ios::app);

  if (!outputFile) {
    std::cerr << "Error: Could not open message.txt for appending\n";
    return 1;
  }

  outputFile << "This line was added using append mode.\n";
  outputFile.close();

  std::cout << "New line appended successfully.\n";
  return 0;
}
```

Expected Console Output

```
New line appended successfully.
```

Key Learning

- Use `std::ios::app` to preserve existing data.
- Without append mode, normal output mode may overwrite file content.

Modification Task

Ask the user for a name and append the name with the current date entered by the user.

Concept 4: Copy One Text File into Another

Aim

To copy the content of one text file into another text file.

Program

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
  std::ifstream sourceFile("message.txt");
  std::ofstream destinationFile("message_copy.txt");

  if (!sourceFile) {
```

```
    std::cerr << "Error: Could not open source file.\n";
    return 1;
  }

  if (!destinationFile) {
    std::cerr << "Error: Could not create destination file.\n";
    return 1;
  }

  std::string line;
  while (std::getline(sourceFile, line)) {
    destinationFile << line << '\n';
  }

  std::cout << "File copied successfully to message_copy.txt\n";
  return 0;
}
```

Expected Console Output

```
File copied successfully to message_copy.txt
```

Key Learning

- An input stream reads the source.
- An output stream writes the destination.
- The copy is line based, suitable for text files.

Modification Task

Copy only lines containing the word `C++` into a new file named `cpp_lines.txt`.

Concept 5: Count Lines, Words, and Characters

Aim

To count lines, words, and characters in a text file.

Program

```
#include <cctype>
#include <fstream>
#include <iostream>
#include <string>

int main() {
  std::ifstream inputFile("message.txt");
```

```
  if (!inputFile) {
    std::cerr << "Error: Could not open message.txt\n";
    return 1;
  }

  std::size_t lineCount = 0;
  std::size_t wordCount = 0;
  std::size_t characterCount = 0;
  bool insideWord = false;

  char ch;
  while (inputFile.get(ch)) {
    ++characterCount;

    if (ch == '\n') {
      ++lineCount;
    }

    if (std::isspace(static_cast<unsigned char>(ch))) {
      insideWord = false;
    } else if (!insideWord) {
      ++wordCount;
      insideWord = true;
    }
  }

  if (characterCount > 0) {
    inputFile.clear();
    inputFile.seekg(-1, std::ios::end);
    char lastCharacter;
    inputFile.get(lastCharacter);
    if (lastCharacter != '\n') {
      ++lineCount;
    }
  }

  std::cout << "Lines: " << lineCount << '\n';
  std::cout << "Words: " << wordCount << '\n';
  std::cout << "Characters: " << characterCount << '\n';

  return 0;
}
```

Expected Output

The exact values depend on the content of `message.txt`.

```
Lines: 4
Words: 25
Characters: 150
```

Key Learning

- `get(ch)` reads one character at a time.
- `std::isspace()` helps identify spaces, tabs, and newline characters.
- File statistics are commonly used in text-analysis tools.

Modification Task

Count vowels, consonants, digits, spaces, and punctuation characters.

Concept 6: Search for a Word in a File

Aim

To search for a word and count its occurrences in a text file.

Program

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
  std::ifstream inputFile("message.txt");

  if (!inputFile) {
    std::cerr << "Error: Could not open message.txt\n";
    return 1;
  }

  std::string searchWord;
  std::cout << "Enter word to search: ";
  std::cin >> searchWord;

  std::string word;
  int count = 0;

  while (inputFile >> word) {
    if (word == searchWord) {
      ++count;
```

```
    }
  }

  std::cout << "The word '" << searchWord << "' occurred " << count << " time(s).\n";
  return 0;
}
```

Sample Input

```
Enter word to search: C++
```

Sample Output

```
The word 'C++' occurred 1 time(s).
```

Important Note

This simple program matches exact whitespace-separated words. Punctuation such as `C++`. or `file`, may affect matching. A more advanced version can normalize punctuation and case.

Modification Task

Make the search case-insensitive and ignore punctuation marks.

Concept 7: Store Student Records in a Text File

Aim

To write student records into a text file using a simple delimiter format.

Record Format

```
rollNumber|name|marks
```

Example:

```
101|Amit Patil|85.5
```

Program

```
#include <fstream>
#include <iostream>
#include <limits>
#include <string>

int main() {
  std::ofstream outputFile("students.txt", std::ios::app);

  if (!outputFile) {
```

```
    std::cerr << "Error: Could not open students.txt\n";
    return 1;
  }

  int rollNumber;
  std::string name;
  double marks;

  std::cout << "Enter roll number: ";
  std::cin >> rollNumber;

  std::cout << "Enter name: ";
  std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
  std::getline(std::cin, name);

  std::cout << "Enter marks: ";
  std::cin >> marks;

  outputFile << rollNumber << '|' << name << '|' << marks << '\n';

  std::cout << "Student record saved successfully.\n";
  return 0;
}
```

Sample Input

```
Enter roll number: 101
Enter name: Amit Patil
Enter marks: 85.5
```

File Content

```
101|Amit Patil|85.5
```

Modification Task

Add course name and mobile number fields to the record.

Concept 8: Read and Search Student Records

Aim

To read delimiter-separated student records and search by roll number.

Program

```cpp
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>

int main() {
  std::ifstream inputFile("students.txt");

  if (!inputFile) {
    std::cerr << "Error: Could not open students.txt\n";
    return 1;
  }

  int targetRollNumber;
  std::cout << "Enter roll number to search: ";
  std::cin >> targetRollNumber;

  std::string line;
  bool found = false;

  while (std::getline(inputFile, line)) {
    std::stringstream record(line);
    std::string rollText;
    std::string name;
    std::string marksText;

    if (std::getline(record, rollText, '|') &&
      std::getline(record, name, '|') &&
      std::getline(record, marksText)) {

      int rollNumber = std::stoi(rollText);
      double marks = std::stod(marksText);

      if (rollNumber == targetRollNumber) {
        std::cout << "Record Found\n";
        std::cout << "Roll Number: " << rollNumber << '\n';
        std::cout << "Name: " << name << '\n';
        std::cout << "Marks: " << marks << '\n';
        found = true;
        break;
      }
    }
  }
```

```
    if (!found) {
        std::cout << "Student record not found.\n";
    }

    return 0;
}
```

Key Learning

- `std::stringstream` helps parse a line into fields.
- `std::stoi()` converts text to integer.
- `std::stod()` converts text to double.

Modification Task

Display all student records in a formatted table.

Concept 9: Update a Student Record Using a Temporary File

Aim

To update a file record safely using a temporary file.

Concept

A common file-update workflow is:

```
Read original file
    ↓
Write unchanged/updated records to temporary file
    ↓
Close both files
    ↓
Replace original file with temporary file
```

Program

```
#include <cstdio>
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>

int main() {
    std::ifstream inputFile("students.txt");
    std::ofstream temporaryFile("students_temp.txt");
```

```
if (!inputFile || !temporaryFile) {
   std::cerr << "Error: Could not open file(s).\n";
   return 1;
}

int targetRollNumber;
double updatedMarks;

std::cout << "Enter roll number to update: ";
std::cin >> targetRollNumber;

std::cout << "Enter updated marks: ";
std::cin >> updatedMarks;

std::string line;
bool found = false;

while (std::getline(inputFile, line)) {
   std::stringstream record(line);
   std::string rollText;
   std::string name;
   std::string marksText;

   if (std::getline(record, rollText, '|') &&
      std::getline(record, name, '|') &&
      std::getline(record, marksText)) {

      int rollNumber = std::stoi(rollText);

      if (rollNumber == targetRollNumber) {
         temporaryFile << rollNumber << '|' << name << '|' << updatedMarks << '\n';
         found = true;
      } else {
         temporaryFile << line << '\n';
      }
   }
}

inputFile.close();
temporaryFile.close();

if (!found) {
   std::remove("students_temp.txt");
```

```
    std::cout << "Student record not found. No update performed.\n";
    return 0;
  }

  if (std::remove("students.txt") != 0) {
    std::cerr << "Error: Could not remove old students.txt\n";
    return 1;
  }

  if (std::rename("students_temp.txt", "students.txt") != 0) {
    std::cerr << "Error: Could not rename temporary file.\n";
    return 1;
  }

  std::cout << "Student marks updated successfully.\n";
  return 0;
}
```

Key Learning

- Text files are commonly updated through a temporary file.
- Always close files before renaming/removing them.
- In production systems, use robust backup and error-recovery methods.

Modification Task

Update both name and marks of a selected student.

Concept 10: File Pointer Navigation

Aim

To use file pointers for finding positions and navigating in a file.

Concept

- `tellg()` reports the current input position.
- `tellp()` reports the current output position.
- `seekg()` moves the input position.
- `seekp()` moves the output position.

Program

```
#include <fstream>
#include <iostream>
#include <string>
```

```
int main() {
   std::fstream file("navigation.txt", std::ios::in | std::ios::out | std::ios::trunc);

   if (!file) {
      std::cerr << "Error: Could not open navigation.txt\n";
      return 1;
   }

   file << "ABCDE";
   std::cout << "Output position after writing: " << file.tellp() << '\n';

   file.flush();
   file.seekg(0, std::ios::beg);

   char firstCharacter;
   file.get(firstCharacter);
   std::cout << "First character: " << firstCharacter << '\n';
   std::cout << "Input position after reading one character: " << file.tellg() << '\n';

   file.seekg(2, std::ios::beg);
   char thirdCharacter;
   file.get(thirdCharacter);
   std::cout << "Character at position 2: " << thirdCharacter << '\n';

   file.seekp(5, std::ios::beg);
   file << "F";
   file.close();

   std::cout << "Navigation completed. Check navigation.txt\n";
   return 0;
}
```

Expected Console Output

```
Output position after writing: 5
First character: A
Input position after reading one character: 1
Character at position 2: C
Navigation completed. Check navigation.txt
```

Expected File Content

```
ABCDEF
```

Key Learning

- Positions are measured in bytes/characters for this simple text example.
- Text-mode seeking can have platform-specific considerations; fixed-size records in binary files are the most common teaching use of random access.

Modification Task

Use `seekg()` to read the last character from `navigation.txt`.

Concept 11: Binary File Writing and Reading

Aim

To write and read fixed-size records in a binary file.

Important Note

Binary `read()` and `write()` examples are safest with simple fixed-size data structures. Avoid writing complex objects containing `std::string`, pointers, or dynamically allocated memory directly as raw bytes.

Program

```
#include <cstring>
#include <fstream>
#include <iostream>

struct StudentRecord {
  int rollNumber;
  char name[30];
  float marks;
};

int main() {
  StudentRecord student{};
  student.rollNumber = 101;
  std::strncpy(student.name, "Amit Patil", sizeof(student.name) - 1);
  student.marks = 85.5F;

  {
    std::ofstream outputFile("students.dat", std::ios::binary);

    if (!outputFile) {
      std::cerr << "Error: Could not create students.dat\n";
      return 1;
    }

    outputFile.write(reinterpret_cast<const char*>(&student), sizeof(student));
  }
```

```
    StudentRecord readStudent{};

    {
        std::ifstream inputFile("students.dat", std::ios::binary);

        if (!inputFile) {
            std::cerr << "Error: Could not open students.dat\n";
            return 1;
        }

        inputFile.read(reinterpret_cast<char*>(&readStudent), sizeof(readStudent));

        if (!inputFile) {
            std::cerr << "Error: Could not read record from students.dat\n";
            return 1;
        }
    }

    std::cout << "Roll Number: " << readStudent.rollNumber << '\n';
    std::cout << "Name: " << readStudent.name << '\n';
    std::cout << "Marks: " << readStudent.marks << '\n';

    return 0;
}
```

Expected Output

```
Roll Number: 101
Name: Amit Patil
Marks: 85.5
```

Key Learning

- Use `std::ios::binary` for binary files.
- `write()` stores raw bytes.
- `read()` retrieves raw bytes.
- This technique is suitable for simple POD-like fixed-size records used in introductory demonstrations.

Modification Task

Write three student records into the binary file and read all of them using a loop.

Concept 12: Random Access in a Binary File

Aim

To access a selected fixed-size record directly from a binary file.

Program

```
#include <cstring>
#include <fstream>
#include <iostream>

struct StudentRecord {
   int rollNumber;
   char name[30];
   float marks;
};

void addRecord(std::ofstream& file, int rollNumber, const char* name, float marks) {
   StudentRecord student{};
   student.rollNumber = rollNumber;
   std::strncpy(student.name, name, sizeof(student.name) - 1);
   student.marks = marks;
   file.write(reinterpret_cast<const char*>(&student), sizeof(student));
}

int main() {
   {
      std::ofstream outputFile("records.dat", std::ios::binary | std::ios::trunc);

      if (!outputFile) {
         std::cerr << "Error: Could not create records.dat\n";
         return 1;
      }

      addRecord(outputFile, 101, "Amit", 85.5F);
      addRecord(outputFile, 102, "Neha", 91.0F);
      addRecord(outputFile, 103, "Ravi", 78.0F);
   }

   std::ifstream inputFile("records.dat", std::ios::binary);

   if (!inputFile) {
      std::cerr << "Error: Could not open records.dat\n";
      return 1;
   }

   int recordNumber;
```

```
    std::cout << "Enter record number to read (1 to 3): ";
    std::cin >> recordNumber;

    if (recordNumber < 1 || recordNumber > 3) {
        std::cerr << "Invalid record number.\n";
        return 1;
    }

    const std::streamoff offset = static_cast<std::streamoff>(recordNumber - 1) *
                                  static_cast<std::streamoff>(sizeof(StudentRecord));

    inputFile.seekg(offset, std::ios::beg);

    StudentRecord selectedStudent{};
    inputFile.read(reinterpret_cast<char*>(&selectedStudent), sizeof(selectedStudent));

    if (!inputFile) {
        std::cerr << "Error: Could not read selected record.\n";
        return 1;
    }

    std::cout << "Roll Number: " << selectedStudent.rollNumber << '\n';
    std::cout << "Name: " << selectedStudent.name << '\n';
    std::cout << "Marks: " << selectedStudent.marks << '\n';

    return 0;
}
```

Sample Input

```
Enter record number to read (1 to 3): 2
```

Expected Output

```
Roll Number: 102
Name: Neha
Marks: 91
```

Formula Used

```
offset = (recordNumber - 1) × sizeof(StudentRecord)
```

Modification Task

Search a binary file by roll number instead of record number.

Concept 13: File Error Handling and Stream States

Aim

To check and report file errors using stream-state functions.

Stream-State Functions

Function	Meaning
good()	No error flags are set
eof()	End of file has been reached
fail()	Logical input/output failure occurred
bad()	Serious input/output error occurred

Program

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
  std::ifstream inputFile("missing_file.txt");

  if (!inputFile.is_open()) {
    std::cerr << "Error: File could not be opened.\n";
    std::cerr << "Check whether missing_file.txt exists in the current folder.\n";
    return 1;
  }

  std::string line;
  while (std::getline(inputFile, line)) {
    std::cout << line << '\n';
  }

  if (inputFile.eof()) {
    std::cout << "End of file reached normally.\n";
  } else if (inputFile.bad()) {
    std::cerr << "A serious file I/O error occurred.\n";
  } else if (inputFile.fail()) {
    std::cerr << "A logical file read error occurred.\n";
  }

  return 0;
```

```
}
```

Expected Output When the File Does Not Exist

```
Error: File could not be opened.
Check whether missing_file.txt exists in the current folder.
```

Key Learning

- Check whether a file stream opens successfully before reading/writing.
- Test stream states after operations when robust error handling is required.
- `eof()` becomes true only after an attempted read passes the end of file.

Modification Task

Ask the user for a file name and continue prompting until a valid file is opened.

Concept 14: File Statistics Mini-Project

Aim

To create a file-analysis program that counts lines, words, characters, vowels, digits, and spaces.

Program

```
#include <cctype>
#include <fstream>
#include <iostream>
#include <string>

bool isVowel(char ch) {
  ch = static_cast<char>(std::tolower(static_cast<unsigned char>(ch)));
  return ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u';
}

int main() {
  std::string fileName;
  std::cout << "Enter file name: ";
  std::getline(std::cin, fileName);

  std::ifstream inputFile(fileName);

  if (!inputFile) {
    std::cerr << "Error: Could not open " << fileName << '\n';
    return 1;
  }
```

```
std::size_t lines = 0;
std::size_t words = 0;
std::size_t characters = 0;
std::size_t vowels = 0;
std::size_t digits = 0;
std::size_t spaces = 0;
bool insideWord = false;

char ch;
while (inputFile.get(ch)) {
   ++characters;

   if (ch == '\n') {
      ++lines;
   }

   if (std::isspace(static_cast<unsigned char>(ch))) {
      if (ch == ' ') {
         ++spaces;
      }
      insideWord = false;
   } else if (!insideWord) {
      ++words;
      insideWord = true;
   }

   if (std::isalpha(static_cast<unsigned char>(ch)) && isVowel(ch)) {
      ++vowels;
   }

   if (std::isdigit(static_cast<unsigned char>(ch))) {
      ++digits;
   }
}

if (characters > 0) {
   inputFile.clear();
   inputFile.seekg(-1, std::ios::end);
   char lastCharacter;
   inputFile.get(lastCharacter);
   if (lastCharacter != '\n') {
      ++lines;
   }
}
```

```
    std::cout << "\nFile Statistics\n";
    std::cout << "Lines: " << lines << '\n';
    std::cout << "Words: " << words << '\n';
    std::cout << "Characters: " << characters << '\n';
    std::cout << "Vowels: " << vowels << '\n';
    std::cout << "Digits: " << digits << '\n';
    std::cout << "Spaces: " << spaces << '\n';

    return 0;
}
```

Suggested Test File Content

```
C++ File Handling 2026
Students learn file streams.
```

Expected Result

The program displays file statistics based on the selected input file.

Extension Tasks

1. Count consonants and punctuation characters.
2. Display the most frequently occurring character.
3. Save the statistics report into `report.txt`.

Concept 15: Student Record Manager Mini-Project

Aim

To create a simple file-based student record manager using text files.

Features

- Add student record.
- Display all student records.
- Search by roll number.
- Update marks.

Program

```
#include <cstdio>
#include <fstream>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>
```

```
void addStudent() {
    std::ofstream outputFile("student_records.txt", std::ios::app);

    if (!outputFile) {
        std::cerr << "Error: Could not open student_records.txt\n";
        return;
    }

    int rollNumber;
    std::string name;
    double marks;

    std::cout << "Enter roll number: ";
    std::cin >> rollNumber;

    std::cout << "Enter name: ";
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    std::getline(std::cin, name);

    std::cout << "Enter marks: ";
    std::cin >> marks;

    outputFile << rollNumber << '|' << name << '|' << marks << '\n';
    std::cout << "Record added successfully.\n";
}

void displayStudents() {
    std::ifstream inputFile("student_records.txt");

    if (!inputFile) {
        std::cout << "No student record file found.\n";
        return;
    }

    std::string line;
    std::cout << "\nRoll No.\tName\t\tMarks\n";
    std::cout << "----------------------------------------\n";

    while (std::getline(inputFile, line)) {
        std::stringstream record(line);
        std::string rollText;
        std::string name;
        std::string marksText;
```

```
    if (std::getline(record, rollText, '|') &&
        std::getline(record, name, '|') &&
        std::getline(record, marksText)) {
        std::cout << rollText << "\t\t" << name << "\t\t" << marksText << '\n';
    }
  }
}

void searchStudent() {
  std::ifstream inputFile("student_records.txt");

  if (!inputFile) {
    std::cout << "No student record file found.\n";
    return;
  }

  int targetRoll;
  std::cout << "Enter roll number to search: ";
  std::cin >> targetRoll;

  std::string line;
  bool found = false;

  while (std::getline(inputFile, line)) {
    std::stringstream record(line);
    std::string rollText;
    std::string name;
    std::string marksText;

    if (std::getline(record, rollText, '|') &&
        std::getline(record, name, '|') &&
        std::getline(record, marksText)) {
        if (std::stoi(rollText) == targetRoll) {
            std::cout << "Record Found\n";
            std::cout << "Roll Number: " << rollText << '\n';
            std::cout << "Name: " << name << '\n';
            std::cout << "Marks: " << marksText << '\n';
            found = true;
            break;
        }
    }
  }
```

```
    if (!found) {
        std::cout << "Student not found.\n";
    }
}

void updateMarks() {
    std::ifstream inputFile("student_records.txt");
    std::ofstream temporaryFile("student_records_temp.txt");

    if (!inputFile || !temporaryFile) {
        std::cerr << "Error: Could not open record file(s).\n";
        return;
    }

    int targetRoll;
    double newMarks;

    std::cout << "Enter roll number to update: ";
    std::cin >> targetRoll;
    std::cout << "Enter new marks: ";
    std::cin >> newMarks;

    std::string line;
    bool found = false;

    while (std::getline(inputFile, line)) {
        std::stringstream record(line);
        std::string rollText;
        std::string name;
        std::string marksText;

        if (std::getline(record, rollText, '|') &&
            std::getline(record, name, '|') &&
            std::getline(record, marksText)) {
            if (std::stoi(rollText) == targetRoll) {
                temporaryFile << rollText << '|' << name << '|' << newMarks << '\n';
                found = true;
            } else {
                temporaryFile << line << '\n';
            }
        }
    }

    inputFile.close();
```

```
    temporaryFile.close();

    if (!found) {
        std::remove("student_records_temp.txt");
        std::cout << "Student not found. No changes made.\n";
        return;
    }

    if (std::remove("student_records.txt") != 0 ||
        std::rename("student_records_temp.txt", "student_records.txt") != 0) {
        std::cerr << "Error: Could not replace the record file.\n";
        return;
    }

    std::cout << "Marks updated successfully.\n";
}

int main() {
    int choice;

    do {
        std::cout << "\nStudent Record Manager\n";
        std::cout << "1. Add Student\n";
        std::cout << "2. Display All Students\n";
        std::cout << "3. Search Student\n";
        std::cout << "4. Update Marks\n";
        std::cout << "0. Exit\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;

        switch (choice) {
            case 1:
                addStudent();
                break;
            case 2:
                displayStudents();
                break;
            case 3:
                searchStudent();
                break;
            case 4:
                updateMarks();
                break;
            case 0:
```

```
            std::cout << "Exiting program.\n";
            break;
        default:
            std::cout << "Invalid choice. Try again.\n";
    }
  } while (choice != 0);

  return 0;
}
```

Suggested Extensions

1. Validate marks between 0 and 100.
2. Prevent duplicate roll numbers.
3. Add delete-record operation.
4. Add grade calculation.
5. Store course and department details.

Concept 16: Library Record Mini-Project

Aim

To implement a simple object-oriented library record application using file storage.

Program

```
#include <fstream>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>

class Book {
private:
  int bookId;
  std::string title;
  std::string author;
  bool issued;

public:
  Book(int id, std::string bookTitle, std::string bookAuthor, bool issueStatus = false)
    : bookId(id), title(std::move(bookTitle)), author(std::move(bookAuthor)), issued(issueStatus) {}

  int getBookId() const {
    return bookId;
```

```
    }

    std::string toFileRecord() const {
        return std::to_string(bookId) + "|" + title + "|" + author + "|" + (issued ? "1" : "0");
    }

    void display() const {
        std::cout << "Book ID: " << bookId << '\n';
        std::cout << "Title: " << title << '\n';
        std::cout << "Author: " << author << '\n';
        std::cout << "Status: " << (issued ? "Issued" : "Available") << '\n';
    }
};

void addBook() {
    int id;
    std::string title;
    std::string author;

    std::cout << "Enter book ID: ";
    std::cin >> id;

    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    std::cout << "Enter title: ";
    std::getline(std::cin, title);

    std::cout << "Enter author: ";
    std::getline(std::cin, author);

    Book book(id, title, author);

    std::ofstream outputFile("library_books.txt", std::ios::app);
    if (!outputFile) {
        std::cerr << "Error: Could not open library_books.txt\n";
        return;
    }

    outputFile << book.toFileRecord() << '\n';
    std::cout << "Book added successfully.\n";
}

void displayBooks() {
    std::ifstream inputFile("library_books.txt");
```

```
    if (!inputFile) {
        std::cout << "No library record file found.\n";
        return;
    }

    std::string line;
    while (std::getline(inputFile, line)) {
        std::stringstream record(line);
        std::string idText;
        std::string title;
        std::string author;
        std::string issuedText;

        if (std::getline(record, idText, '|') &&
            std::getline(record, title, '|') &&
            std::getline(record, author, '|') &&
            std::getline(record, issuedText)) {
            Book book(std::stoi(idText), title, author, issuedText == "1");
            book.display();
            std::cout << "-------------------------\n";
        }
    }
}

int main() {
    int choice;

    do {
        std::cout << "\nLibrary Record System\n";
        std::cout << "1. Add Book\n";
        std::cout << "2. Display Books\n";
        std::cout << "0. Exit\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;

        switch (choice) {
            case 1:
                addBook();
                break;
            case 2:
                displayBooks();
                break;
            case 0:
```

```
            std::cout << "Exiting program.\n";
            break;
        default:
            std::cout << "Invalid choice.\n";
    }
  } while (choice != 0);

  return 0;
}
```

Concepts Used

- Class and objects.
- Text-file storage.
- `ofstream` and `ifstream`.
- Serialization using delimiter-separated records.
- Menu-driven programming.
- Error handling.

Suggested Extensions

1. Search a book by ID.
2. Issue and return books by updating status.
3. Prevent duplicate book IDs.
4. Add a student borrower record.
5. Add due date and fine calculation.

Quick Reference Sheet

Create/write a file

```
std::ofstream outputFile("data.txt");
if (!outputFile) {
  // Handle error
}
outputFile << "Text";
```

Read a file

```
std::ifstream inputFile("data.txt");
std::string line;
while (std::getline(inputFile, line)) {
  // Process line
}
```

Append to a file

```
std::ofstream outputFile("data.txt", std::ios::app);
```

Read and write with one stream

```
std::fstream file("data.txt", std::ios::in | std::ios::out);
```

File positions

```
file.tellg();
file.tellp();
file.seekg(position, std::ios::beg);
file.seekp(position, std::ios::beg);
```

Binary read/write

```
file.write(reinterpret_cast<const char*>(&record), sizeof(record));
file.read(reinterpret_cast<char*>(&record), sizeof(record));
```

Stream state checks

```
file.good();
file.eof();
file.fail();
file.bad();
```

Common Errors and Fixes

Problem	Likely cause	Fix
File does not open	Incorrect path or insufficient permission	Verify file name, folder, and permissions
Old content disappears	File opened in normal output/truncate mode	Use std::ios::app to append
Extra loop iteration	Using while (!file.eof())	Use while (std::getline(file, line)) or successful read condition
File cannot be renamed	File stream still open	Close all streams before remove/rename
Search fails with punctuation	Exact word comparison	Normalize case and remove punctuation
Binary record is corrupted	Writing non-trivial objects such as std::string as raw bytes	Use fixed-size character arrays or proper serialization
Update did not work	Temporary file replacement failed	Check remove() and rename() results
Read starts from wrong location	File pointer not reset	Use clear() and seekg() as needed